

Cryptography - 25_11

Today, we'll talk about **RSA**. This was the original **PKE construction** from 1978.

The motivation was to build PKE on the hardness of **factoring**.

The basic idea:

- $KGen(1^\lambda)$: $sk = (n, p, q, d)$ such that $n = p \cdot q$ ($|p| \approx |q| \approx \lambda$ bits). $pk = (n, e)$.
With $e \cdot d = 1 \pmod{\phi(n)}$, such that $\phi(n) = (p-1)(q-1)$.

Basically we choose an e , which could be random or fixed.

Typical choice: e is a prime (e.g. $e = 3$).

- $Enc(pk, m \in \mathbb{Z}_n^*) : c = m^e \pmod{n}$
- $Dec(sk, c) : c^d \pmod{n}$
$$c^d = (m^e)^d = m^{e \cdot d} \equiv m^{t \cdot \phi(n) + 1} \equiv \underbrace{(m^{\phi(n)})^t}_1 \cdot m \equiv m \pmod{n}$$

Of course, not CPA secure! Because it is deterministic.

Solution: Encode m into $\hat{m}$ in a randomized (also invertible) fashion.

- PKCS # 1.5. Basically says $\hat{m} = (m || r)$, where r is a random bit string.

Is it provably secure? Under which assumption and for what size $|r|$?

Clearly $|r|$ cannot be too short ($O(\log \lambda)$), otherwise we can just guess it.

Unfortunately, for real-world values of r we can't prove RSA with PKCS # 1.5 padding CPA secure.

On the other hand, if $m \in \{0, 1\}$ and all the rest is a random r , then it can be proven CPA secure. Under which assumption?

✓ Definition (RSA Assumption)

Let $GenRSA \rightarrow (pk, sk)$ be the $KGen$ in RSA. Then, for all PPT $\mathcal{A}$:

$$Pr[\mathcal{A}(y, n, e) = x : x \leftarrow \mathbb{Z}_n^*; y = x^e \pmod{n}; pk = (n, e);$$
$$sk = (n, d); \text{ using } GenRSA(1^\lambda)] \leq \text{negl}(\lambda)$$

Equivalently: $f_{n,e}(x) = f_{pk}(x)$ is a OWF.

What is the relation between RSA and factoring? $RSA \implies$ factoring.

Because given p, q we can compute d and recover $x = y^d \pmod n$.

Other direction is unknown, at least for general algorithms.

- PKCS # 2.0. A different encoding that suffices for CCA security.

What this is basically is a two 2-round Feistel network using hash functions G, H starting with $m||r$, with r is uniform.

The proof requires RSA assumption plus the assumption that G and H are random oracles.

What really RSA is? A trapdoor permutation (TDP): $(KGen, f, f^{-1})$ such that $(pk, sk) \leftarrow KGen(1^\lambda)$

$$f_{pk} : \mathbf{X} \rightarrow \mathbf{X}$$

$$f_{sk}^{-1} : \mathbf{X} \rightarrow \mathbf{X}$$

$$\forall x \in \mathbf{X} : f_{sk}^{-1}(f_{pk}(x)) = x$$

(RSA: $pk = (n, e), sk = (n, d)$)

$$f_{pk}(x) = x^e \pmod n = y$$

$$f_{sk}^{-1}(y) = y^d \pmod n$$

Security-wise, f_{pk} is a one way permutation even given pk .

Fact

$$TDP \implies PKE$$

Let $(KGen, f, f^{-1})$ be a TDP. Let h be hard-core for f .

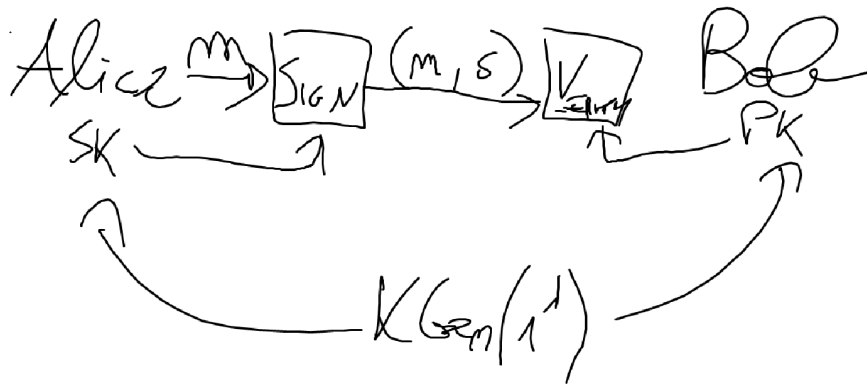
$$Enc(pk, m \in \{0, 1\}) : (f_{pk}(r), h_{pk}(r) \oplus m) \quad r \leftarrow \mathbf{X}$$

Note: PKE for one bit $\implies$ PKE for *poly* bits.

Here is what we plan to do next:

- Digital signatures: FDH (RSA) and Fiat-Shamir (DL, ...)
- Post-quantum crypto, lattices for PKE and Digital Signature
- Exercises
- Bonus: CCA security for PKE, zero knowledge proofs.

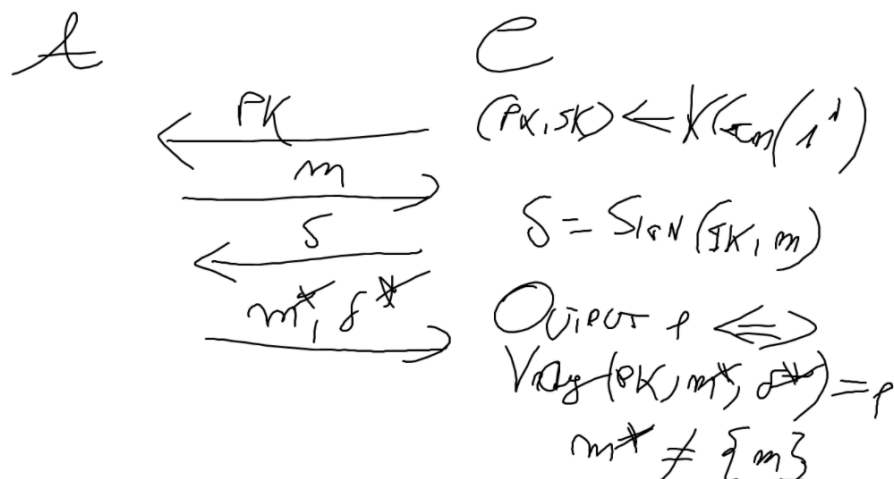
Let's take a look at **Digital Signatures**.



✓ Definition (UFCMA)

We say $\Pi = (KGen, Sign, Vrfy)$ is UFCMA if $\forall$ PPT $\mathcal{A}$:

$$Pr[GAME_{\Pi, \mathcal{A}}^{ufcma}(\lambda) = 1] \leq \text{negl}(\lambda)$$



Most important application: **PKI** (Public Key Infrastructure) and X.509 certificates.

Simplest construction: **RSA Signatures**.

- $KGen(1^\lambda) : pk = (n, e); sk = (n, d)$ as in RSA.
- $Sign(sk, m) : \sigma = m^d \pmod{n}$
- $Vrfy(pk, m, \sigma) : \text{Check } \sigma^e = m \pmod{n}$

This is not UF-CMA! Why? Signature of m , i.e. $\sigma = m^d \pmod{n}$

$\sigma^2 = (m^d)^2 = (m^2)^d \pmod{n}$, valid signature on $m^* = m^2$.

Or, pick any σ and let $m = \sigma^e \pmod{n}$.

🔗 Exercise

Forge on any chosen $m^* \in \mathbb{Z}_n^*$

How do we fix this? We use **Full-Domain Hash** (FDH), by which we hash the message first.

$$\sigma = H(m)^d \pmod{n}$$

More in general: Use a TDP $(KGen, f, f^{-1})$. $Sign(sk, m) = f_{sk}^{-1}(y)$ $y = H(m)$.

$Vrfy(pk, m, \sigma)$: Check $f_{pk}(\sigma) = y = H(m)$.

✓ Theorem

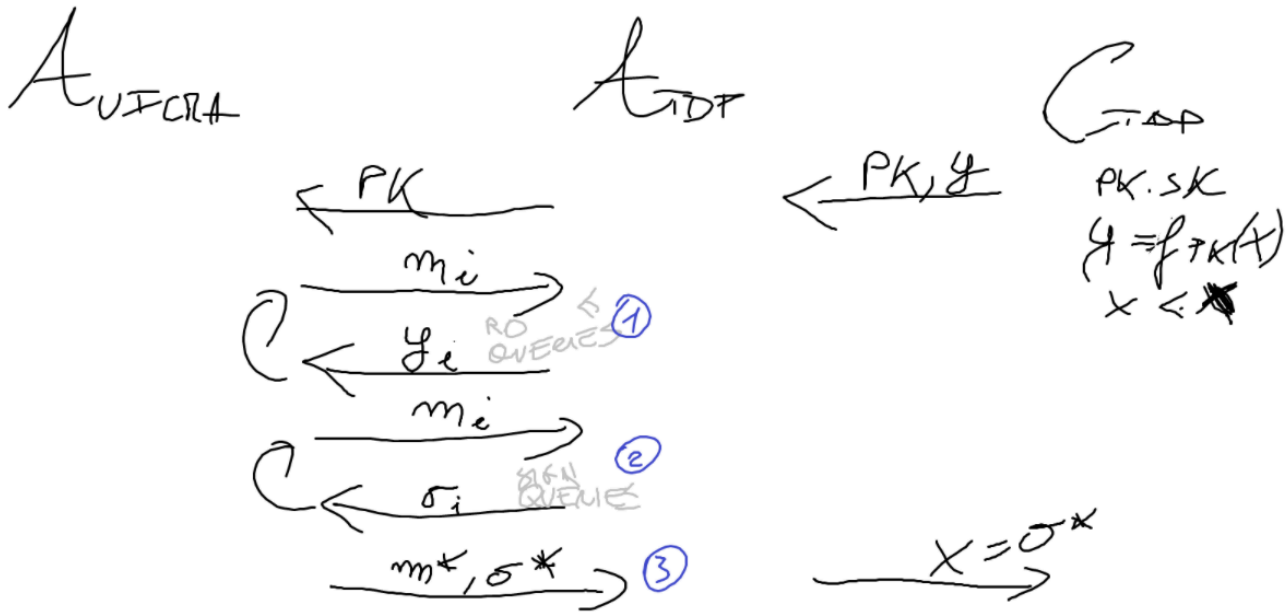
Assuming the TDP is one-way, FDH is UFCMA in the Random Oracle Model (ROM).

🔗 Proof

Main trick: In the ROM, the attacker will ask random oracle (RO) queries. This will also help simulating signature queries. We make some assumptions without loss of generality:

- Let $q_h = \#RO$ queries, $q_s = \#signature$ queries
- $\mathcal{A}$ never repeats queries.
- Before asking signature on m_i , or forging m^* , $\mathcal{A}$ will ask to RO the query m_i/m^* . So if he asks for RO of m_i , he will also ask the signature query for m_i

Here is the reduction:



Let's start with [3]. If σ^* is valid, then σ^* is a pre-image of $y^* = H(m^*)$.

Main idea: Try to enforce that $H(m^*) = y^* = y$. We want to enforce that $\hat{y} = y^*$. While we don't know when m^* was asked to the RO, we can guess it.

At the beginning pick: $j \leftarrow [q_h]$.

For [1], upon input a RO query m_i :

- If $i = j$, return $H(m_i) = y$. Basically we pick one query among the polynomial queries, cheating, and sending the adversary something that is not random like other values, and the adversary has no way of knowing this (???, per chance).
- If $i \neq j$, sample $x_j \leftarrow \mathbf{X}$ and output $y_i = H(m_i) = f_{pk}(x_i)$

For [2], upon m_i return x_i corresponding to y_i .

For [3] (again), when the forgery comes m^*, σ^* , output $\sigma^* = x$.

Assuming j is guessed correctly:

- The simulation is perfect. Which means that the public key is perfectly distributed, RO queries are uniform (first choosing the pre-image, then permuting), and also signature queries are handled correctly. Random oracle queries and sign queries are correctly distributed.
- With probability $\frac{1}{poly}$, σ^* is a pre-image of $H(m^*) = y$.

So,

$$Pr[\mathcal{A}_{TDP} \text{ wins}] = Pr[\text{Guessing } j] \times Pr[\mathcal{A}_{ufcma} \text{ wins}] \geq \frac{1}{poly(\lambda)}$$

